

11.2 Freeing Memory

Memory allocated with `malloc` lasts as long as you want it to. It does not automatically disappear when a function returns, as automatic-duration variables do, but it does not have to remain for the entire duration of your program, either. Just as you can use `malloc` to control exactly when and how much memory you allocate, you can also control exactly when you deallocate it.

In fact, many programs use memory on a transient basis. They allocate some memory, use it for a while, but then reach a point where they don't need that particular piece any more. Because memory is not inexhaustible, it's a good idea to deallocate (that is, release or *free*) memory you're no longer using.

Dynamically allocated memory is deallocated with the `free` function. If `p` contains a pointer previously returned by `malloc`, you can call

```
free(p);
```

which will "give the memory back" to the stock of memory (sometimes called the "arena" or "pool") from which `malloc` requests are satisfied. Calling `free` is sort of the ultimate in recycling: it costs you almost nothing, and the memory you give back is immediately usable by other parts of your program. (Theoretically, it may even be usable by other programs.)

(Freeing unused memory is a good idea, but it's not mandatory. When your program exits, any memory which it has allocated but not freed should be automatically released. If your computer were to somehow "lose" memory just because your program forgot to free it, that would indicate a problem or deficiency in your operating system.)

Naturally, once you've freed some memory you must remember not to use it any more. After calling

```
free(p);
```

it is probably the case that `p` still points at the same memory. However, since we've given it back, it's now "available," and a later call to `malloc` might give that memory to some other part of your program. If the variable `p` is a global variable or will otherwise stick around for a while, one good way to record the fact that it's not to be used any more would be to set it to a null pointer:

```
free(p);  
p = NULL;
```

Now we don't even have the pointer to the freed memory any more, and (as long as we check to see that `p` is non-NULL before using it), we won't misuse any memory via the pointer `p`.

When thinking about `malloc`, `free`, and dynamically-allocated memory in general, remember again the distinction between a pointer and what it points to. If you call `malloc` to allocate some memory, and store the pointer which `malloc` gives you in a local pointer variable, what happens when the function containing the local pointer variable returns? If the local pointer variable has *automatic duration* (which is the default, unless the variable is declared `static`), it will disappear when the function returns. But for the pointer variable to disappear says nothing about the memory pointed to! That memory still exists and, as far as `malloc` and `free` are concerned, is still allocated. The only thing that has disappeared is the pointer variable you had which pointed at the allocated memory. (Furthermore, if it contained the only copy of the pointer you had, once it disappears, you'll have no way of freeing the memory, and no way of using it, either. Using memory and freeing memory both require that you have at least one pointer to the memory!)

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995-1997 // [mail](#) [feedback](#)